학습 목표

프레임워크 없이 루프를 짜 보고, 추상화가 가리는 것을 확인한다

1. **Messages API로 tool_use/tool_result 순환을 처리하는 에이전트 루프를 직접 작성할 수 있다**
   40여 줄짜리 run_agent 함수 하나로 구현

2. **max_turns, 비용 상한, 시간 상한을 동시에 적용하고 각 종료 사유를 구분해 반환할 수 있다**
   completed, max_turns, budget_exceeded, timeout 등을 구분

3. **stop_reason(end_turn, tool_use, max_tokens, refusal)별 분기 처리를 구현할 수 있다**
   예상하지 못한 값은 unexpected로 처리

4. **직접 구현, tool_runner, Agent SDK query()의 차이를 비교표로 정리하고 상황에 맞는 수준을 고를 수 있다**
   대신해 주는 것과 가리는 것을 함께 비교

5. **Agent SDK의 ResultMessage subtype(error_max_turns, error_max_budget_usd)을 분기 처리할 수 있다**
   resumable 여부와 session_id를 함께 기록

무단전재 및 배포 금지

밤새 돌아간 헬프데스크 에이전트

문제는 모델이 아니라 모델 바깥의 루프 코드, 곧 하네스였다

WHY 사고 상황과 원인	상한 없는 while True 루프로 티켓 200건 야간 배치를 걸었습니다. 검색 도구가 "다시 검색하세요"를 돌려줄 때마다 모델이 같은 도구를 수백 번 불렀고, 반복 횟수, 비용, 시간 어느 것에도 상한이 없었습니다.
HOW 이 장의 접근	API를 직접 호출하는 40여 줄 루프 함수부터 만들고, tool_runner와 Agent SDK로 다시 구현해 각 추상화가 대신하는 것과 가리는 것을 확인합니다.
WHAT 핵심 결과	턴 수, 누적 비용, 경과 시간에 정지 조건을 걸고, 루프가 왜 끝났는지 종료 사유로 기록해 어떤 결과를 믿어도 되는지 가립니다.

> ***정지 조건과 종료 사유 기록은 하네스의 책임이다***

출처: Anthropic Engineering, "Building effective agents" (2024-12-19)

무단전재 및 배포 금지

에이전트 루프의 한 바퀴

정지 조건 검사부터 tool_result 추가까지 한 바퀴를 되풀이한다

1 정지 조건 검사
턴 수, 누적 비용, 경과 시간 가운데 하나라도 한도를 넘으면 해당 종료 사유로 멈춥니다.

2 Messages API 호출
지금까지의 대화 전체를 messages 배열로 보냅니다. 모델이 맡는 단계는 이것 하나뿐입니다.

3 stop_reason 분기
end_turn이면 completed, max_tokens 또는 refusal이면 각 사유로 종료하고, tool_use면 다음 단계로 갑니다.

4 도구 실행
응답의 tool_use 블록마다 등록된 도구를 실행합니다. 예외는 is_error 결과로 바꿉니다.

5 tool_result 추가
결과를 user 메시지로 붙이고 다시 정지 조건 검사로 돌아갑니다.

> ***모델 호출을 뺀 모든 단계가 하네스의 책임이다***

출처: Anthropic, "How the agent loop works (Agent SDK)", Claude Code Docs

무단전재 및 배포 금지

메시지 히스토리 누적 규칙

API는 상태를 저장하지 않으므로 매 턴 전체 히스토리를 다시 보낸다

json

```
[
 {"role": "user",
  "content": "티켓 T-1024의 원인과 해결 방법을 알려 주세요."},
 {"role": "assistant", "content": [
  {"type": "text", "text": "먼저 티켓 내용을 확인하겠습니다."},
  {"type": "tool_use", "id": "toolu_01A",
   "name": "get_ticket", "input": {"ticket_id": "T-1024"}}
 ]},
 {"role": "user", "content": [
  {"type": "tool_result", "tool_use_id": "toolu_01A",
   "content": "{\"status\": \"open\", ...}",
   "is_error": false}
 ]}
]
```

1. **turn** = 모델 호출 1회와 그에 따른 도구 실행. 이 장의 비교 기준이다
2. 모델 응답(tool_use 포함)은 **그대로** assistant 메시지로 붙인다
3. 도구 결과는 **user 역할** 메시지에 tool_result로 넣고 tool_use_id로 짝을 맞춘다
4. tool_use가 여러 개면 모든 결과를 **하나의 user 메시지**에 모은다

> ***턴이 늘수록 입력 토큰이 쌓여 비용이 더 빠르게 는다***

무단전재 및 배포 금지

stop_reason별 분기 처리

stop_reason은 모델이 생성을 멈춘 이유이자 루프의 방향이다

stop_reason과 하네스의 처리

stop_reason	의미	하네스의 처리
end_turn	모델이 할 말을 마침	정상 종료(completed)
tool_use	도구 실행을 요청함	실행 후 tool_result를 붙여 다음 턴
max_tokens	한도에 걸려 응답이 잘림	종료 또는 한도를 올려 재요청
refusal	안전상 이유로 응답 거절	거절로 종료, 그대로 반복 안 함
그 밖의 값	stop_sequence 등	unexpected로 종료, 로그 기록

1. 한도 검사는 호출 전에만 하므로 10번째 응답이 end_turn이면 **completed**다
2. 잘린 tool_use는 실행하지 않는다. 재개 전에는 짝 없는 tool_use 블록을 제거한다
3. pause_turn(서버 도구 사용 시): 응답을 붙이고 tool_result 없이 다시 요청해 이어 간다
4. model_context_window_exceeded는 컨텍스트 관리(4장)로 넘길 신호다

> ***어느 정책을 택하든 정책을 코드에 명시한다***

출처: Anthropic, "Python SDK" / Messages API 문서; refusal과 max_tokens 처리는 "Structured outputs", Claude Platform Docs

무단전재 및 배포 금지

루프 구현: 정지 조건 3종

다음 모델 호출을 보내기 전에 세 가지 한도를 먼저 검사한다

python

```
while True:
    if turns >= limits.max_turns:
        return done(StopReason.MAX_TURNS)
    if cost >= limits.max_cost_usd:
        return done(StopReason.BUDGET)
    remaining = limits.max_seconds - (clock() - start)
    if remaining <= 0:
        return done(StopReason.TIMEOUT)
    try:  # 호출 타임아웃도 남은 시간으로 제한
        resp = client.messages.create(
            ..., messages=messages, timeout=remaining)
    except anthropic.APITimeoutError:
        return done(StopReason.TIMEOUT)
```

1. **max_turns**: 같은 도구를 끝없이 재호출하는 사고를 막는다
2. **비용 상한**은 넘은 뒤 멈춘다. 호출 한 번만큼 초과하므로 실제 한도보다 낮게 잡는다
3. **시간 상한**: 남은 시간을 timeout으로 넘기고, max_retries=1로 재시도 대기를 줄인다
4. clock 인자(기본 time.monotonic)를 가짜 시계로 바꾸면 시간 상한을 결정적으로 테스트할 수 있다

> ***completed만 업무에 반영하고 나머지 사유는 재시도 큐로***

출처: Anthropic, "Claude Agent SDK reference — Python", Claude Code Docs (max_budget_usd 초과 성질)

무단전재 및 배포 금지

루프 구현: 도구 실행과 결과

도구 예외는 루프 밖으로 던지지 않고 is_error 결과로 돌려준다

python

```
results = []
for block in resp.content:
    if block.type != "tool_use": continue
    fn = registry.get(block.name)
    if fn is None:
        out, is_err = f"알 수 없는 도구: {block.name}", True
    else:
        try: out, is_err = fn(**block.input), False
        except Exception as e:
            out, is_err = f"{type(e).__name__}: {e}", True
    results.append({
        "type": "tool_result", "tool_use_id": block.id,
        "content": str(out), "is_error": is_err})
messages.append({"role": "user", "content": results})
```

1. is_error 결과를 읽은 모델이 인자를 고치거나 다른 방법을 택할 수 있다
2. 미등록 도구도 예외 없이 오류 결과로 돌려준다(fn is None 분기)
3. 가짜 도구는 KeyError 대신 LookupError를 써서 오류 문자열에 따옴표가 섞이지 않게 한다
4. 모든 결과를 모아 하나의 user 메시지로 붙인다

무단전재 및 배포 금지

tool_runner로 루프 자동화

러너가 도구 루프를 돌리고, 한도와 종료 사유는 직접 붙인다

python

```
for message in runner:          # 순회 1회 = 1턴
    st["turns"] += 1
    st["cost"] += cost_of(message.usage)
    if message.stop_reason != "tool_use":   # 마지막 응답
        st["reason"] = STOP_MAP.get(message.stop_reason,
                                    "unexpected")
        break
    tool_response = runner.generate_tool_call_response()
    if st["turns"] >= LIMITS.max_turns:   # 다음 호출 전 검사
        st["reason"] = "max_turns"; break
    # 비용과 시간 검사도 같은 방식
```

1. @beta_tool이 타입 힌트와 docstring으로 도구 스키마를 만든다
2. 러너에 맡길 수 있는 상한은 반복 횟수(max_iterations)뿐이다
3. generate_tool_call_response()로 결과를 미리 검사하고 감사 로그를 남긴다
4. timeout은 호출당 고정이므로 전체 시간은 바깥 스레드와 cancel 플래그로 자른다

> ***베타 API이므로 버전을 고정하고 check_sdk.py로 먼저 확인***

출처: Anthropic, "Python SDK (Anthropic Client SDK)", Claude Platform Docs

무단전재 및 배포 금지

Agent SDK 옵션과 도구 통제

ClaudeAgentOptions로 한도를 걸고, 노출 도구를 두 단계로 검사한다

python

```
EXPECTED_TOOLS = {"mcp__helpdesk__search_docs",
                  "mcp__helpdesk__get_ticket"}
MCP_AUX_TOOLS = {"ListMcpResourcesTool",
                 "ReadMcpResourceTool"}
options = ClaudeAgentOptions(
    model=MODEL, mcp_servers={"helpdesk": server},
    allowed_tools=sorted(EXPECTED_TOOLS),  # 자동 승인일 뿐
    disallowed_tools=BUILTIN_TOOLS + sorted(MCP_AUX_TOOLS),
    max_turns=LIMITS.max_turns,
    max_budget_usd=LIMITS.max_cost_usd)  # 기본값은 무제한
# init 메시지의 노출 도구를 두 단계로 검사
missing = EXPECTED_TOOLS - tools       # 필요한 도구 누락?
extra = tools - EXPECTED_TOOLS - MCP_AUX_TOOLS  # 허용 밖?
# missing 또는 extra가 있으면 ToolExposureError
```

1. tools=[]로 내장 도구 세트를 비운다(필드가 있는 버전만)
2. allowed_tools는 자동 승인일 뿐 차단이 아니며, 차단은 disallowed_tools가 맡는다
3. 내장 도구 목록은 예시이므로 init 메시지로 실제 노출 도구를 검사한다
4. MCP 보조 도구는 호출을 막고, 목록에 남는 것은 허용 목록으로 통과시킨다

출처: Anthropic, "Claude Agent SDK reference — Python", Claude Code Docs

무단전재 및 배포 금지

ResultMessage 분기 처리

루프가 끝나면 ResultMessage.subtype으로 종료 사유를 분기한다

python

```
def handle_result(r: ResultMessage):
    s = {"reason": r.subtype, "turns": r.num_turns,
         "cost_usd": r.total_cost_usd,
         "session_id": r.session_id}
    if r.subtype == "success":
        s["text"] = r.result
    elif r.subtype in ("error_max_turns",
                       "error_max_budget_usd"):
        s["resumable"] = True   # session_id로 resume(8장)
    else:
        s["resumable"] = False  # 구조화 출력 재시도 초과 등
    return s
```

1. 한도 초과로 끝나도 ResultMessage가 먼저 도착하므로 버리지 않는다
2. 단발 query()는 에러 결과 뒤 예외를 던질 수 있어 try로 감싼다
3. asyncio.wait_for로 취소하면 턴 수, 비용, session_id가 모두 사라진다
4. interrupt()는 루프만 멈추고 ResultMessage를 끝까지 받게 해 준다

> ***벽시계 시간 상한은 옵션이 없으니 바깥에서 건다***

출처: Anthropic, "Claude Agent SDK reference — Python", Claude Code Docs

무단전재 및 배포 금지

세 가지 수준 비교

추상화가 높아질수록 대신해 주는 것과 가리는 것이 함께 는다

직접 구현, tool_runner, Agent SDK query() 비교

항목	직접 구현	tool_runner	Agent SDK
히스토리 조립	직접	자동	자동
stop_reason 분기	모든 값을 직접 분기	STOP_MAP으로 직접 확인	subtype으로 요약
max_turns	직접 검사	max_iterations + 직접 검사	max_turns 옵션
비용 상한	usage로 직접 계산	직접 계산	max_budget_usd(추정치)
시간 상한	남은 시간을 timeout으로	호출당 고정, 바깥 스레드	옵션 없음, interrupt()
측정에 포함	내 요청만	내 요청만	CLI 기동, 덧붙는 지시
가려지는 것	거의 없음	도구 예외 변환, 루프 내부	요청 본문, 턴 계산 방식

1. **직접 구현**: 요청과 응답 원문 감사, 가짜 모델 단위 테스트가 필요할 때
2. **tool_runner**: 도구 몇 개로 끝나는 짧은 작업을 빠르게 만들 때
3. **Agent SDK**: 파일 작업, 긴 세션, 권한, 재개가 필요한 범용 에이전트
4. 턴 수는 버전마다 다를 수 있으니 비교 기록에 함께 적는다

무단전재 및 배포 금지

워크플로 패턴 5종을 루프로

누가 다음 단계를 정하고 어디서 멈추는지가 패턴마다 다르다

루프 관점의 워크플로 패턴

패턴	루프 모양	다음 단계 결정	정지 조건
프롬프트 체이닝	고정 순서로 N번 호출	코드	모든 단계 완료 또는 gate 실패
라우팅	분류 1번 → 처리기 1개	분류 결과와 코드	처리기 종료
병렬화	sectioning 또는 voting 동시 호출	코드	모든 호출 후 취합
오케스트레이터-워커	바깥 LLM이 워커 루프 실행	오케스트레이터 LLM	end_turn과 전체 예산
평가자-최적화자	생성 → 평가 → 재생성	평가 결과	합격 또는 최대 반복

1. 워크플로는 코드 경로가 정해져 있고, 에이전트는 LLM이 다음 단계를 정한다
2. 평가자-최적화자도 비용 누적과 stop_reason 확인이 필요한 루프다
3. 멀티에이전트는 채팅의 약 15배 토큰을 쓰므로 바깥과 워커 루프 모두에 정지 조건을 둔다
4. 가장 단순한 형태에서 시작해 필요할 때만 복잡도를 올린다

출처: Anthropic Engineering, "Building effective agents"; "How we built our multi-agent research system" (2025-06-13)

무단전재 및 배포 금지

핵심 정리

1. **에이전트 루프는 모델 호출을 뺀 모든 단계가 하네스의 책임이다**
2. **매 턴 전체 히스토리를 보내고, tool_result는 tool_use_id로 짝을 맞춰 하나의 user 메시지에 담는다**
3. **end_turn, tool_use, max_tokens, refusal을 분기하고 예상 밖 값은 unexpected로 처리한다**
   잘린 tool_use는 실행하지 않는다
4. **max_turns, 누적 비용, 벽시계 시간 상한을 동시에 걸고 종료 사유를 구분해 반환한다**
5. **비용 상한은 응답 하나만큼 넘을 수 있고, 시간 상한은 남은 시간을 timeout으로 넘겨야 지켜진다**
6. **tool_runner는 루프를 자동화하지만 비용, 시간 상한과 종료 사유 구분은 개발자 몫이다**
7. **Agent SDK는 subtype으로 종료를 알리고, 부분 결과는 interrupt()로 보존한다**
   노출 도구는 두 단계로 검사
8. **베타와 버전 의존 기능은 버전을 고정하고 기능 확인 스크립트를 먼저 돌린다**

무단전재 및 배포 금지

실습 안내

가짜 도구 두 개로 폭주 시나리오를 만들고 세 버전을 비교한다

1 **준비 점검**
check_sdk.py로 OK/MISS를 기록하고 pytest -q test_loop.py로 결정적 검증

2 **정상 경로**
main_direct.py를 실행해 completed와 DOC-001 인용 확인

3 **폭주 시나리오 A/B/C**
HELPDESK_LOOP=1에서 턴, 비용, 시간 상한별로 max_turns / budget_exceeded / timeout 확인

4 **다른 수준으로 반복**
main_runner.py, main_agent_sdk.py로 반복하고 턴, 비용, 토큰, 시간, 종료 사유를 비교표에 기록

> ***일찍 end_turn한 경우는 실패가 아닌 관찰 결과로 기록***

무단전재 및 배포 금지